

머신러닝을 위한  
**Python 기초**

---

**방 수 식 교수**  
(bang@tukorea.ac.kr)

**한국공학대학교 전자공학부**

2024년도 1학기  
머신러닝실습 & 인공지능설계실습1



---

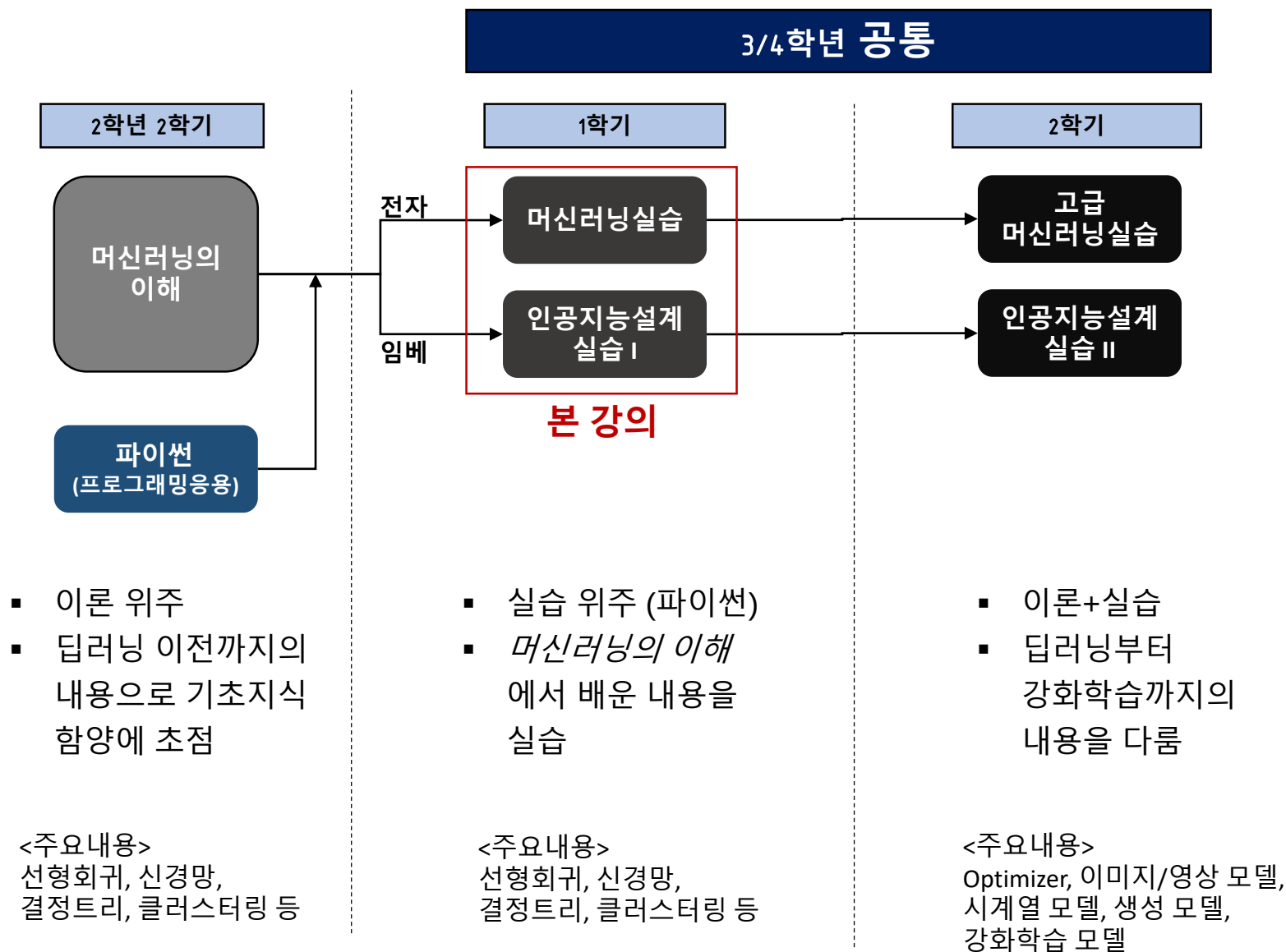
## 교과목 개요

---

# 인공지능 교과목 개요



지능형 예측·진단 연구실  
Intelligent Prognostics & Diagnostics Lab.



# 본 강의 교과목 개요



지능형 예측·진단 연구실  
Intelligent Prognostics & Diagnostics Lab.

- 제목: 머신러닝실습 & 인공지능설계실습1(3학년/4학년 1학기)

- 선수과목: 머신러닝의 이해

- 수업시간: 4시간

- 이론: 1시간, 실습: 3시간

- 실습 시작 후, break time 자유롭게

*퇴실도 가능하나,  
공식 수업시간 이전 퇴실 시  
=> 실습 관련 메일 질의 답변x*

- 담당교수: 방수식

- 산용 509호

- bang@tukorea.ac.kr

*E-class에 쪽지기능은 답변하지 않음.  
반드시 메일로 질문(메일 양식)*

- 교재: 강의PPT자료 및 자체교재 (김준수 교수님 제작)

- E-class 배포예정

# 강의계획 (예정)



지능형 예측·진단 연구실  
Intelligent Prognostics & Diagnostics Lab.

| 주차 | 강의주제                  | 자체 교재   |
|----|-----------------------|---------|
| 1  | Introduction to Class | -       |
| 2  | Python 기초 #1 (기초 문법)  | -       |
| 3  | Python 기초 #2 (데이터 처리) | -       |
| 4  | 선형 회귀                 | Chap. 1 |
| 5  | 모델 검증과 평가             | Chap. 2 |
| 6  | 로지스틱 회귀               | Chap. 3 |
| 7  | 인공 신경망 #1             | Chap. 4 |
| 8  | 중간고사 (프로젝트 공지)        |         |
| 9  | 인공 신경망 #2             | Chap. 4 |
| 10 | 인공 신경망 #3             | Chap. 4 |
| 11 | 의사결정 트리               | Chap. 5 |
| 12 | 클러스터링                 | Chap. 6 |
| 13 | 딥러닝과 강화학습 기초          | 자체 자료   |
| 14 | 프로젝트 발표               | -       |
| 15 | 보강 및 기말고사             |         |
| 16 | 보강 및 기말고사             |         |

- 진도에 따라 유동적으로 변경 가능

## ■ 성적구성

- 출석(10%), 참여도(10%), 실습(35%), 중간프로젝트(15%), 기말고사(30%)
- 출석 점수 구성 (전자출결 기준)
  - 시작 점수: 100, 지각: -10, 결석: -20    => 출퇴 적발 시 2배 가중 적용
- 참여도 점수 구성 (수업 시간 중 수시로 체크)
  - 시작 점수: 50 (만점:100)
  - 창의적인 질문 및 답변: +10 (한정적으로 부여)
  - 태도불량: -10 (지속적인 수업 방해 행위, 수업 외 다른 용도로 PC사용)
- 실습
  - 주차별 시작 점수: 60 (만점:100)
  - 실습 당일: Pass(+10), Non-pass(0), Drop(-20)    => 조교에게 권한 부여
  - 보고서 평가: 1등(+30), 제출(0), 미흡(-10), 미제출(-30)
    - ⇒ 주차별 1등: 학기동안 1회 (중복 수혜 불가)
    - ⇒ 주차별 1등은 학기말 공개

## ■ 성적구성

- 출석(10%), 참여도(10%), 실습(35%), 중간프로젝트(15%), 기말고사(30%)

## ■ 중간/기말고사 (변경 시 e-class 공지)

- 중간고사 (프로젝트)

- 8주차 E-class 온라인 강의로 공지 (출책 O)

- 기말고사 (실기평가)

- 3분반 동시에 평가 진행 – 16주차(6/17~23) 중 예정
- 각 분반별로 상대평가
- 개별 노트북 필수

## ■ 실습 과목 학점부여

### ➤ 학교 가이드라인

- A등급 이상은 30%를 초과할 수 없음
- A등급 및 B등급 이상은 70%를 초과할 수 없음

### ➤ 중요 포인트

- 가이드라인을 준수하면서 성적 분포도에 따라 학점 부여
- 성적 분포도에 따라 30% 이내라도 B를 받을 수 있음.

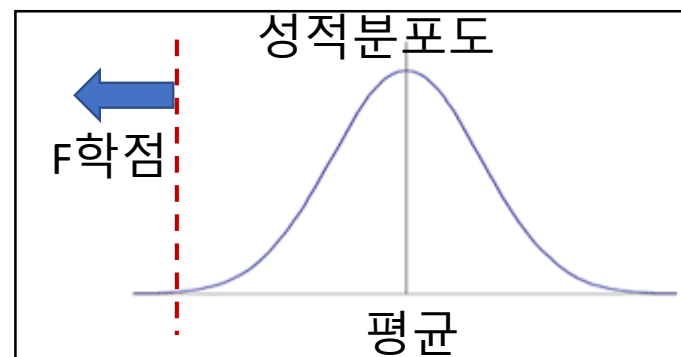
## ■ F학점(낙제) 기준 (아래 중 1가지 사항이라도 해당하는 경우)

### ➤ 1. 출석률 75% 미만

- “2 지각 = 1 결석” => 3결석 1지각까지는 낙제 X

### ➤ 2. 중간고사 또는 기말고사 불참

### ➤ 3. 총점이 정규분포에서 크게 벗어난 경우





## ■ 결석계 제출 인정 기간

- 결석 당일로부터 14일 이내
- 제출 인정 기간 이후 제출은 출석에 반영하지 않음 (본인 책임).

## ■ 결석계 제출 방법

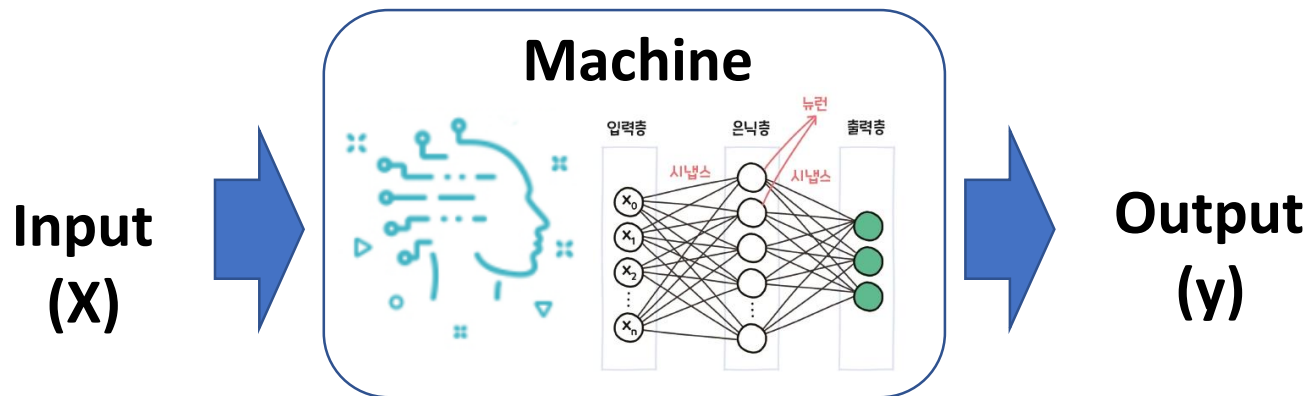
- U-Check 시스템 내 “이의신청”탭 활용
- 학과 발급 출석인정원 등 관련 자료를 반드시 첨부해야만 이의신청 수용
  - 출력본 제출 불필요

# What is “Machine” Learning?



## ■ Physical Meaning

- **Machine:** Input을 받아 Output을 도출하는 일종의 “시스템”
- **Machine Learning:** Machine을 학습시키는 방법론

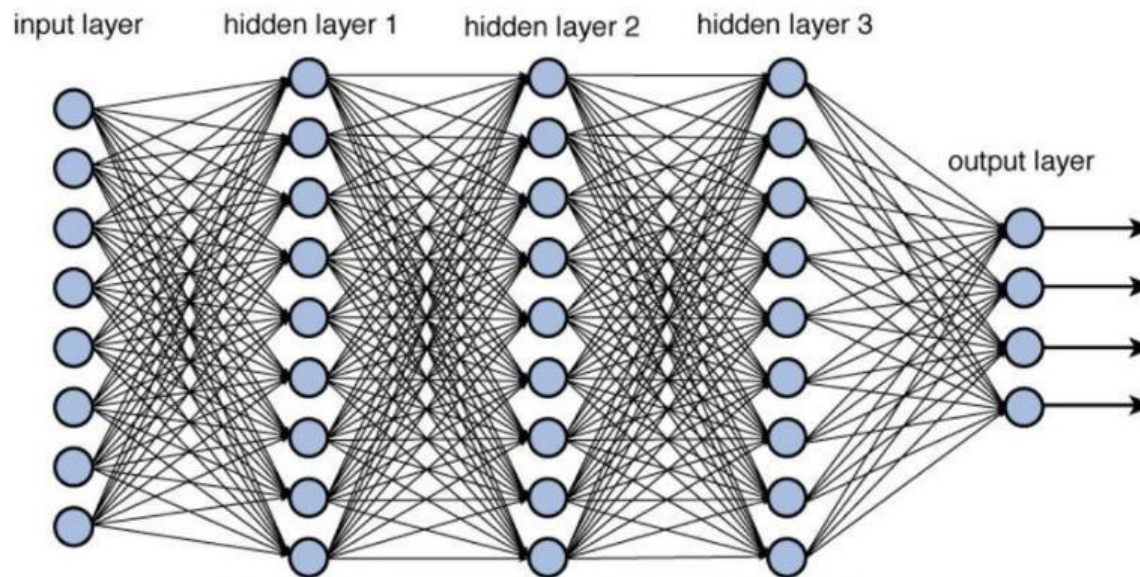


# What is “Deep” Learning?



## ■ Deep Learning

- Machine Learning의 Subfield
- 본래 어원: 신경망이 깊다 (Deep)
- 현재: Input을 별도로 “**Feature Extraction**”하지 않아도 잘 동작하는



## ■ Feature Extraction

- 샘플을 특징 벡터로 feature vector 표현 **Input data**
- 예) Iris 데이터에서 꽃받침의 길이, 꽃받침의 너비, 꽃잎의 길이, 꽃잎의 너비라는 4개의 특징이 각각 5.1, 3.5, 1.4, 0.2인 샘플

$$\mathbf{x} = \begin{pmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \end{pmatrix} = \begin{pmatrix} 5.1 \\ 3.5 \\ 1.4 \\ 0.2 \end{pmatrix}$$



그림 3-2 iris의 세 가지 품종(왼쪽부터 Setosa, Versicolor, Virginica)

- 여러 개의 특징 벡터를 첨자로 구분

$$\mathbf{x}_1 = \begin{pmatrix} 5.1 \\ 3.5 \\ 1.4 \\ 0.2 \end{pmatrix}, \mathbf{x}_2 = \begin{pmatrix} 4.9 \\ 3.0 \\ 1.4 \\ 0.2 \end{pmatrix}, \mathbf{x}_3 = \begin{pmatrix} 4.7 \\ 3.2 \\ 1.3 \\ 0.2 \end{pmatrix}, \dots, \mathbf{x}_{150} = \begin{pmatrix} 5.9 \\ 3.0 \\ 5.1 \\ 1.8 \end{pmatrix}$$

**Input data 1**

**Input data 2**

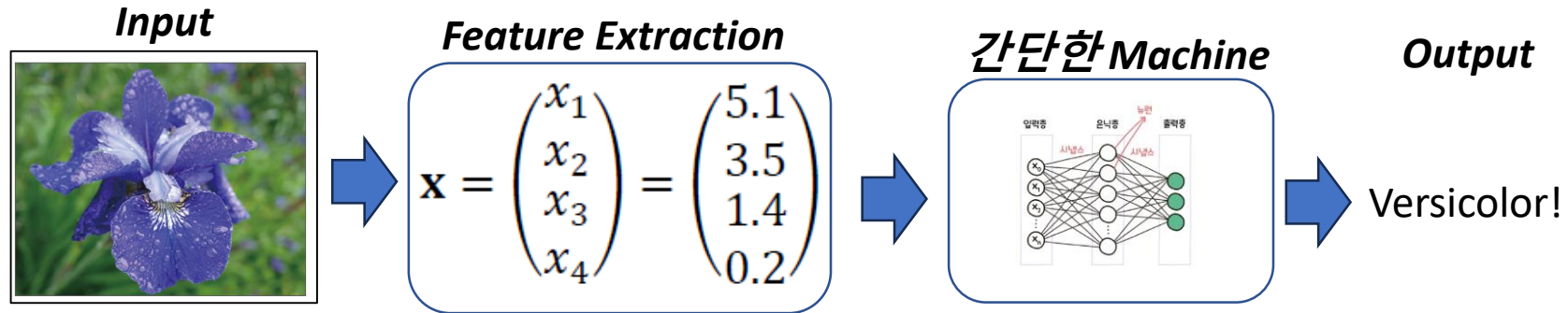
**Input data 3**

# ML vs DL



지능형 예측·진단 연구실  
Intelligent Prognostics & Diagnostics Lab.

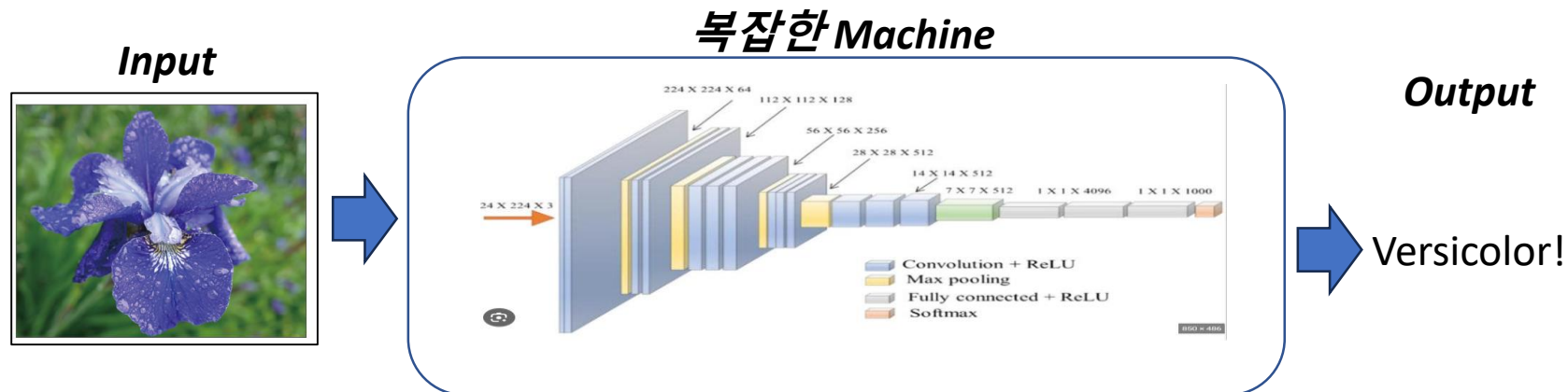
ML



Computing power: 하 (only CPU)

학습의 성공 요건: 특징 추출

DL



Computing power: 상 (Server)

학습의 성공 요건: Network 설계

# Computing Power: 중 (1 GPU)



지능형 예측·진단 연구실  
Intelligent Prognostics & Diagnostics Lab.

Input

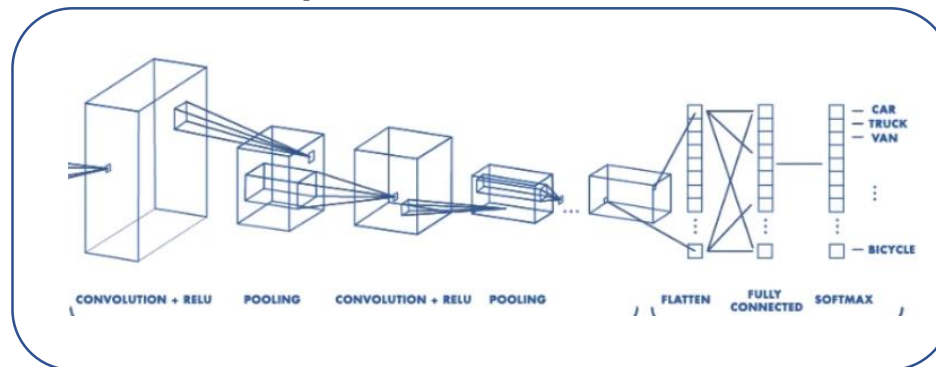


더 많은 Feature Vector 추출

$$\mathbf{x}_1 = \begin{pmatrix} 5.1 \\ 3.5 \\ 1.4 \\ 0.2 \end{pmatrix}, \mathbf{x}_2 = \begin{pmatrix} 4.9 \\ 3.0 \\ 1.4 \\ 0.2 \end{pmatrix}, \mathbf{x}_3 = \begin{pmatrix} 4.7 \\ 3.2 \\ 1.3 \\ 0.2 \end{pmatrix}$$

즉, Machine 성능을 사람이 대신 약간 보완해준다.

약간 복잡한 Machine



Output

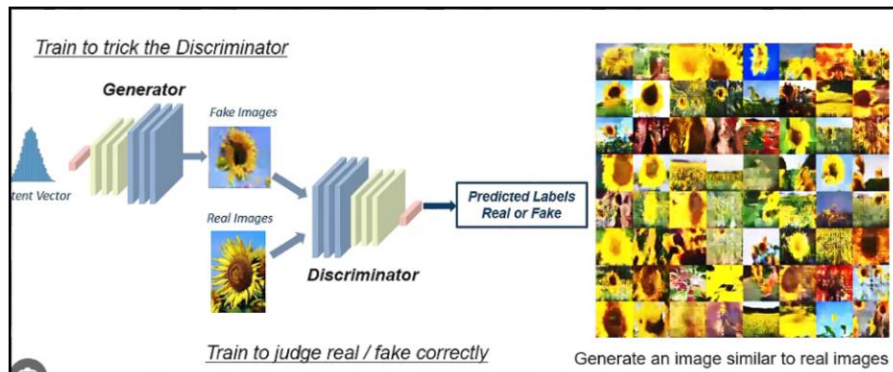
➡ Versicolor!

학습의 성공 요건: 특징 추출도 잘하면서, Network도 잘 설계  
(데이터의 의미를 잘 알아야 한다)

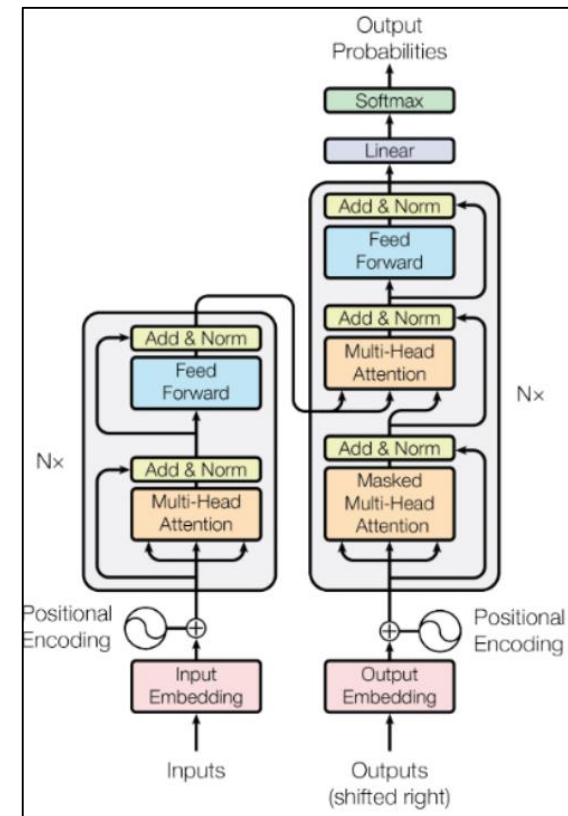


## ■ Network (Architecture, Model) 설계

- CNN, RNN, GAN, Transformer 등 Network의 구조를 새롭게 제안하여 Machine의 성능을 획기적으로 향상시키는 분야



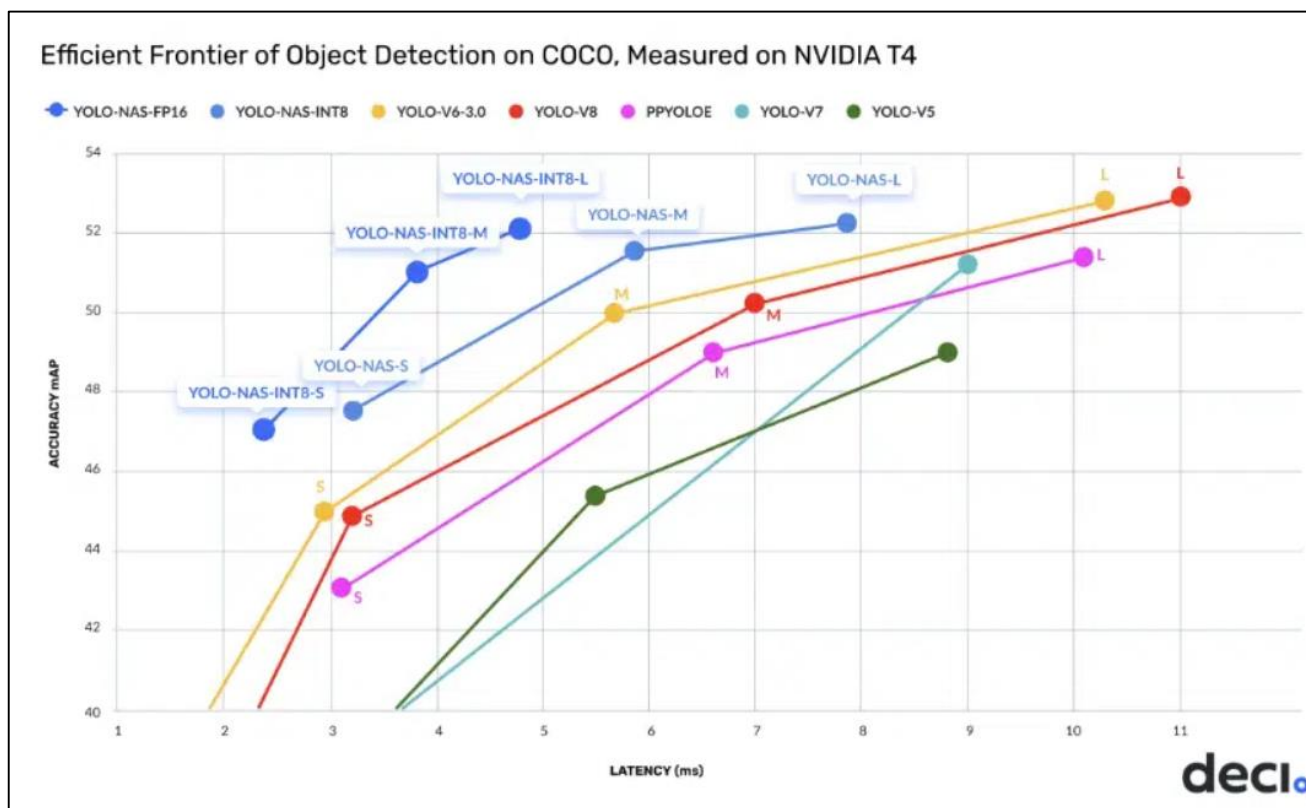
생성형 AI 대표 모델: GAN



자연어처리 대표 모델: Transformer

## ■ Network (Architecture, Model) 최적화

- 새롭게 제안되어 성능이 검증된 Network의 구조를 개선하여 성능을 극대화하는 분야. SOTA(State-of-the-art) 달성이 목표.





## ■ Network (Architecture, Model) 성능 분석

- 발표된 Network을 다양한 data-set에 적용하여, Network을 검증하고 분석하는 분야

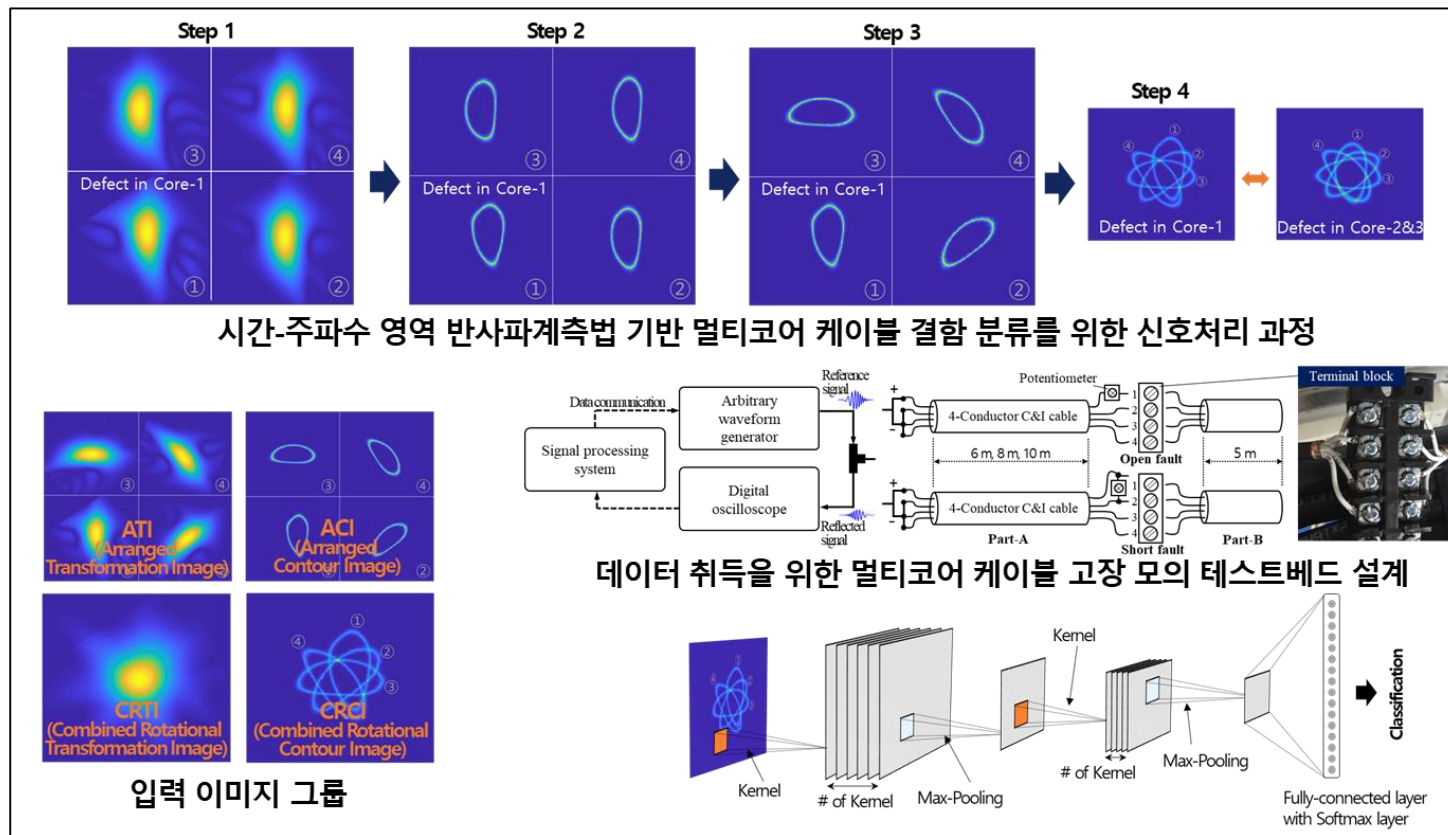
## ■ Training 최적화

- Network을 Training할 때, 사람이 지정해야하는 Hyper-parameter를 최적화하는 분야
- Optimizer (Solver)를 개발하는 분야 등

- Network (Architecture, Model) 응용 (AI+X) **강의자의 분야**
  - 검증 완료되어 오픈소스로 배포된 Network 활용, 산업 도메인의 상황에 맞게 Network와 산업 Data를 최적화하는 분야
  - 도메인에 대한 전문적 지식 필요
  - 다양한 Architecture에 대한 일정 수준 이상의 지식 필요
  - 다양한 도메인에 실제 적용(Data processing 및 Training) 경험 필요

## ■ 멀티코어 케이블(하네스 케이블) 결함 분류

- 도메인: 케이블 진단, 신호 처리
- Architecture: CNN (합성곱 신경망)



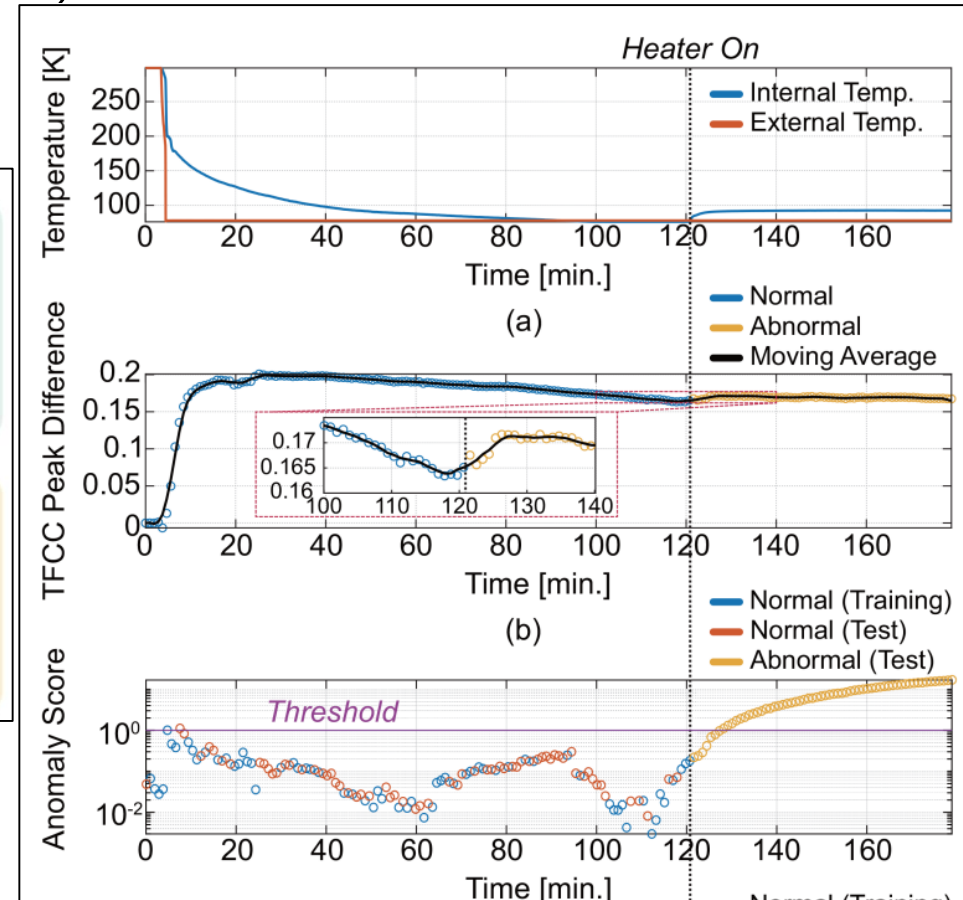
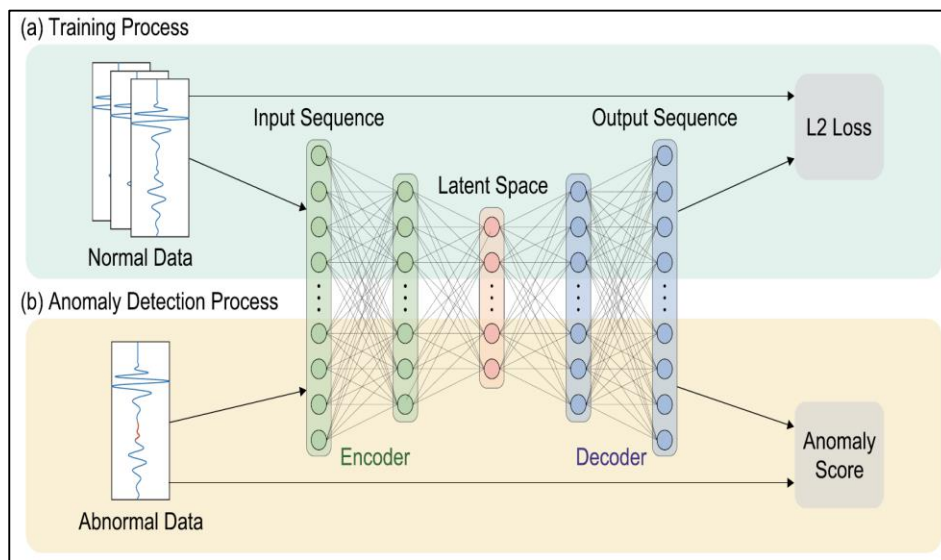
# AI+X 사례 2



지능형 예측·진단 연구실  
Intelligent Prognostics & Diagnostics Lab.

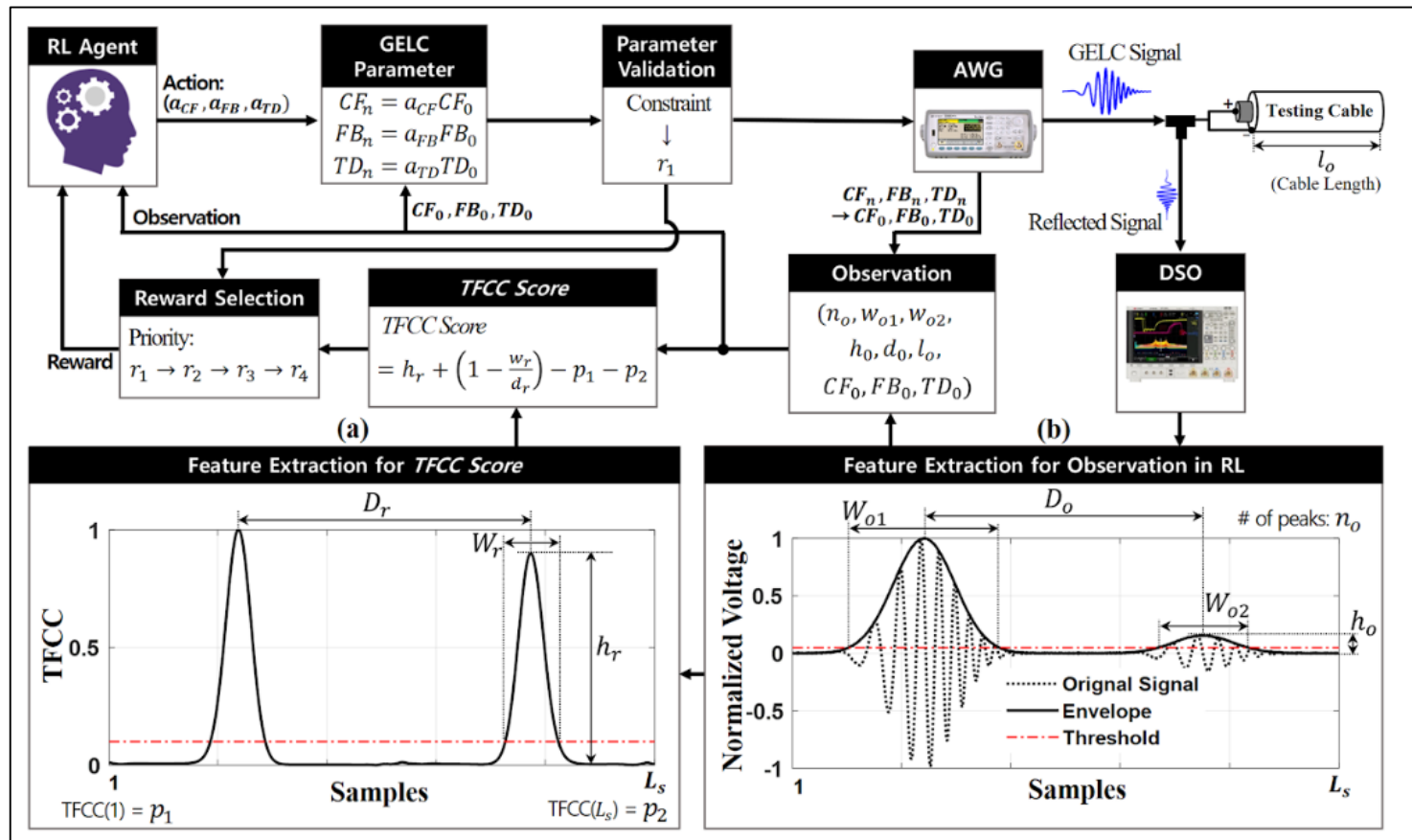
## ■ 케이블 시스템 모니터링

- 도메인: 감시 시스템, 신호 처리
- Architecture: VAE (변분 오토인코더)



## ■ 진단 시스템 자동화

- 도메인: 시스템 자동화, 신호 처리, 최적화
- Architecture: PPO (강화학습)



## ■ Text-to-Image Generation

A rabbit detective sitting on a park bench and reading a newspaper in a victorian setting



Robot dinosaurs versus monster trucks in the colosseum



Ramesh et al., "DALL-E 2." 2022. <https://openai.com/dall-e-2/>

Source: <https://twitter.com/sama/status/1511724264629678084>



## ■ Text-to-Image Generation with better Captions

A whimsical illustration of a rabbit detective dressed in Victorian attire, complete with a trench coat and a magnifying glass tucked into its pocket



A dynamic and action-packed scene set inside the ancient Colosseum, where futuristic robot dinosaurs face off against rugged monster trucks.

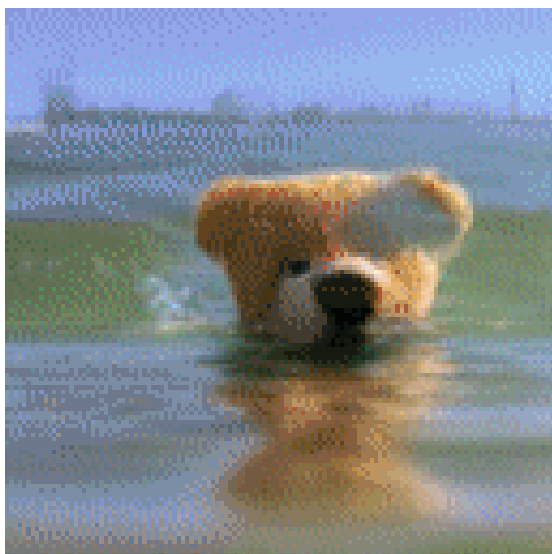


Betker et al., "Improving Image Generation with Better Captions.." 2023

Source: ChatGPT Plus

## ■ Text-to-Video Generation

- A photorealistic teddy bear is swimming in the ocean at San Francisco
- The teddy bear goes under water
- The teddy bear keeps swimming under the water with colorful fishes
- A panda bear is swimming under water



- Side view of an astronaut is walking through a puddle on mars
- The astronaut is dancing on mars
- The astronaut walks his dog on mars
- The astronaut and his dog watch fireworks



Villegas et al., "Phenaki: Variable Length Video Generation From Open Domain Textual Description." ICLR 2023

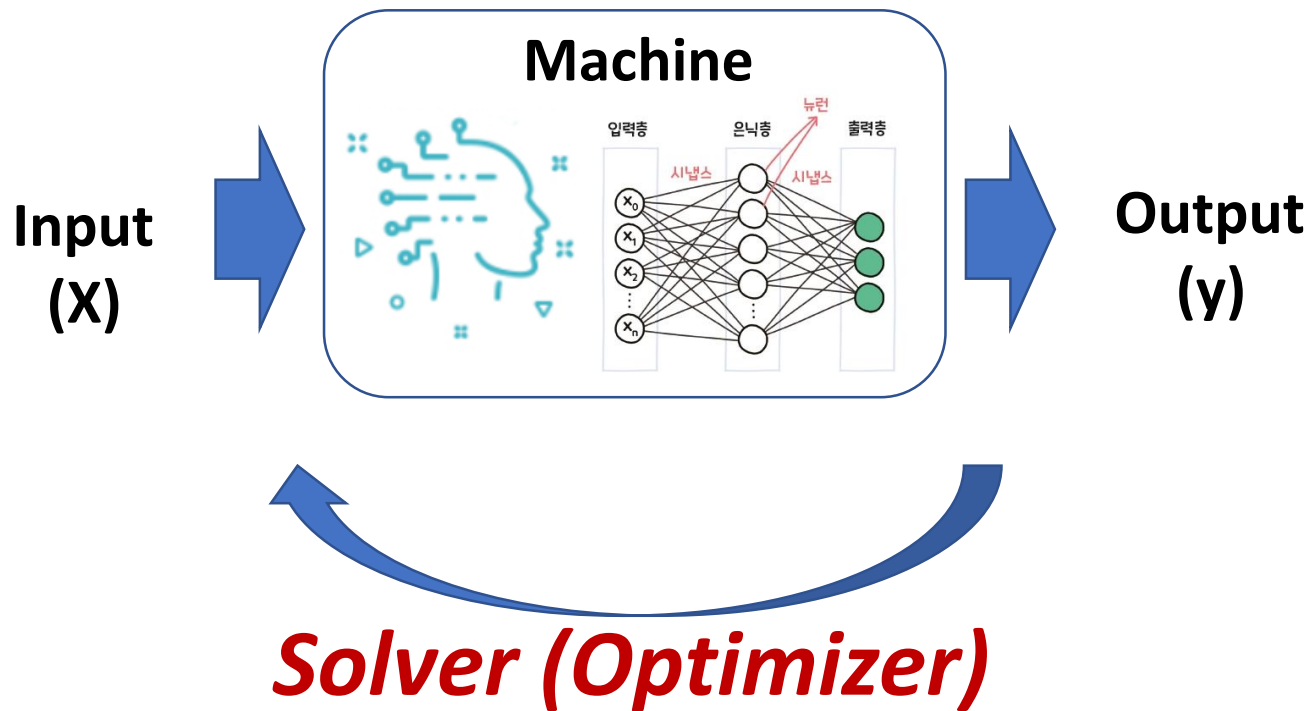
Source: <https://phenaki.research.google/>



# How to “Training” the Machine?



지능형 예측·진단 연구실  
Intelligent Prognostics & Diagnostics Lab.





---

# Python

---

# Python 기반 실습

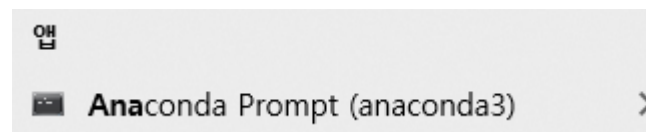
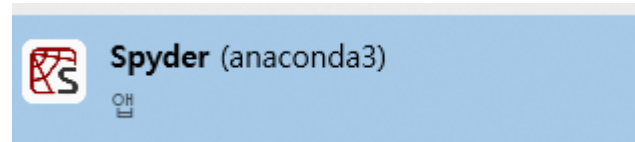


지능형 예측·진단 연구실  
Intelligent Prognostics & Diagnostics Lab.

- 주간 실습
  - Python 기반 실습 진행
- 사용할 Python 환경
  - Anaconda v3.9 통합환경
  - IDE: Spyder
    - ❖ Integrated Development Environment
- 주요 사용 라이브러리 **함수 모음집**
  - NumPy: 수치 연산 및 해석
  - matplotlib: 그래프 표현
  - pandas: 데이터 처리
  - TensorFlow: 인공지능 함수



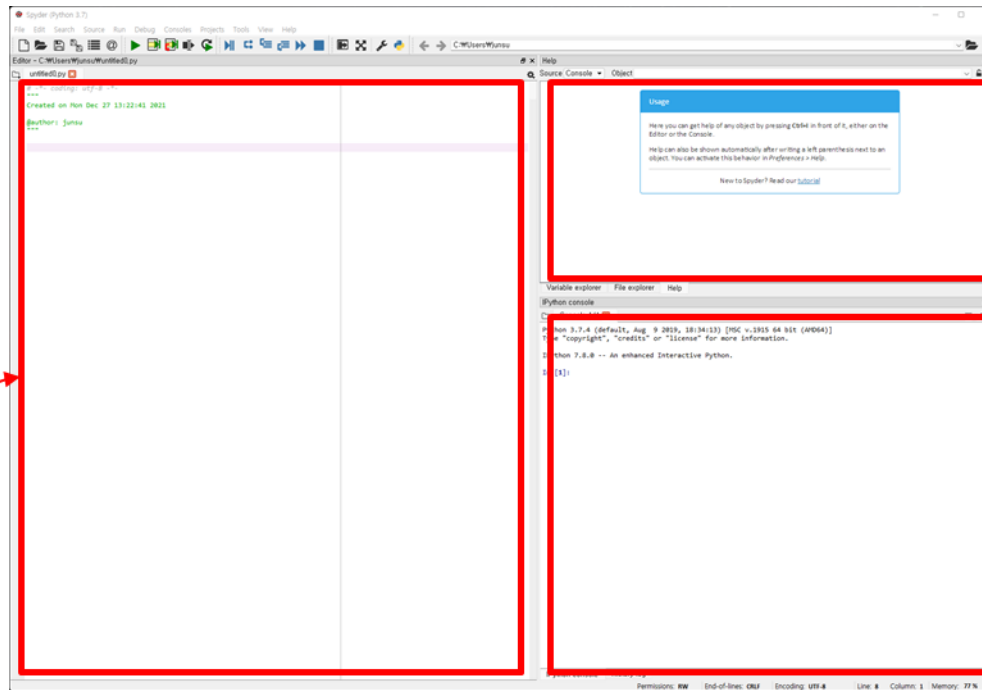
- Anaconda 설치
  - anaconda.com 에서 설치 파일 다운로드 후 설치 진행
    - 대부분의 라이브러리는 자동으로 설치됨
  - Anaconda 코딩 환경은 두 가지 제공
    - Jupyter notebook: 웹브라우저 기반 코딩환경
    - Spyder: 콘솔 기반 코딩환경
- Spyder 실행
  - 탐색기에서 spyder
- Conda prompt 실행
  - 탐색기에서 Anaconda Prompt



# Spyder 실행



지능형 예측·진단 연구실  
Intelligent Prognostics & Diagnostics Lab.



Editor (에디터)  
- 코드 입력 및 편집

정보 표시창  
- 도움말 및 각종 정보

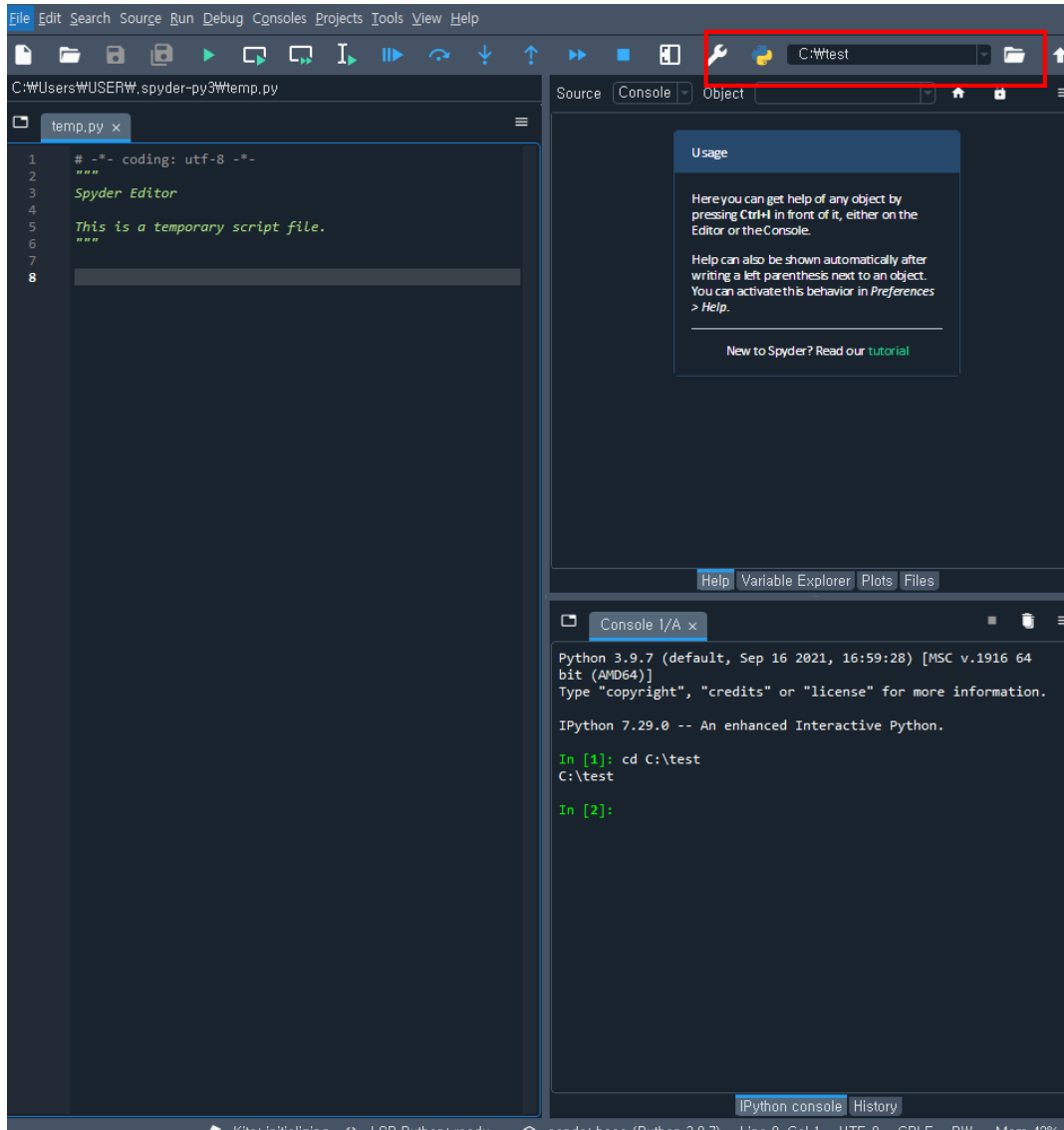
Console (콘솔)  
- 명령 입력  
- 코드 실행 결과 출력

# Directory 설정



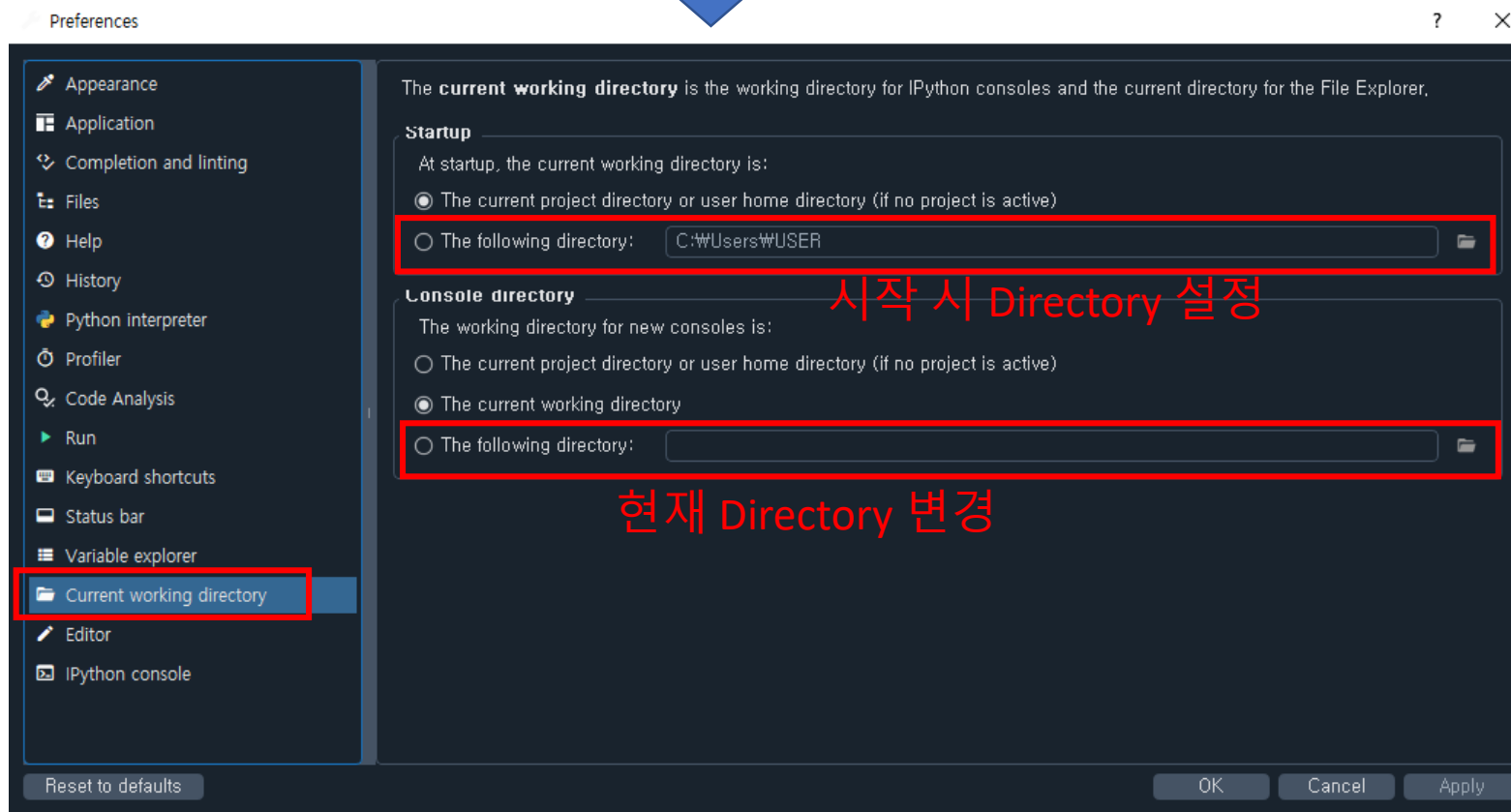
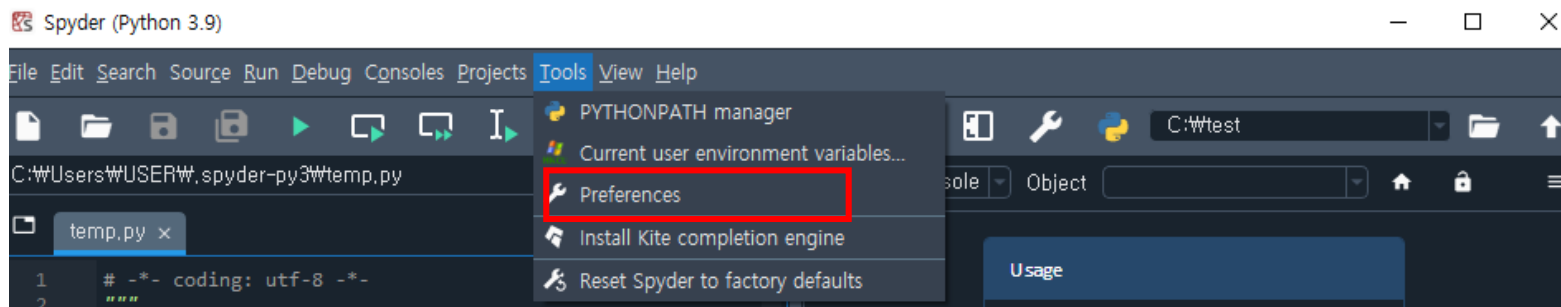
지능형 예측·진단 연구실  
Intelligent Prognostics & Diagnostics Lab.

현재 Directory: 우측 상단에서 변경



Crtl - or + 로  
글자크기 조정 가능

# Directory 설정



# 실행 방법



1. Hello World 를 출력해보자
  2. 1+1 연산을 실행해보자
- Console 창에 명령어 입력 후 Enter
  - Editor에 명령어 입력 후 Run (F5)

```
In [1]: print("Hello World")  
Hello World  
  
In [2]: 1+1  
Out[2]: 2
```

```
temp.py x  
1 print("Hello World")  
2 print(1+1)  
3
```

```
In [3]: runfile('C:/Users/USER/.spyder-py3/temp.py', wdir='C:/  
Users/USER/.spyder-py3')  
Hello World  
2
```

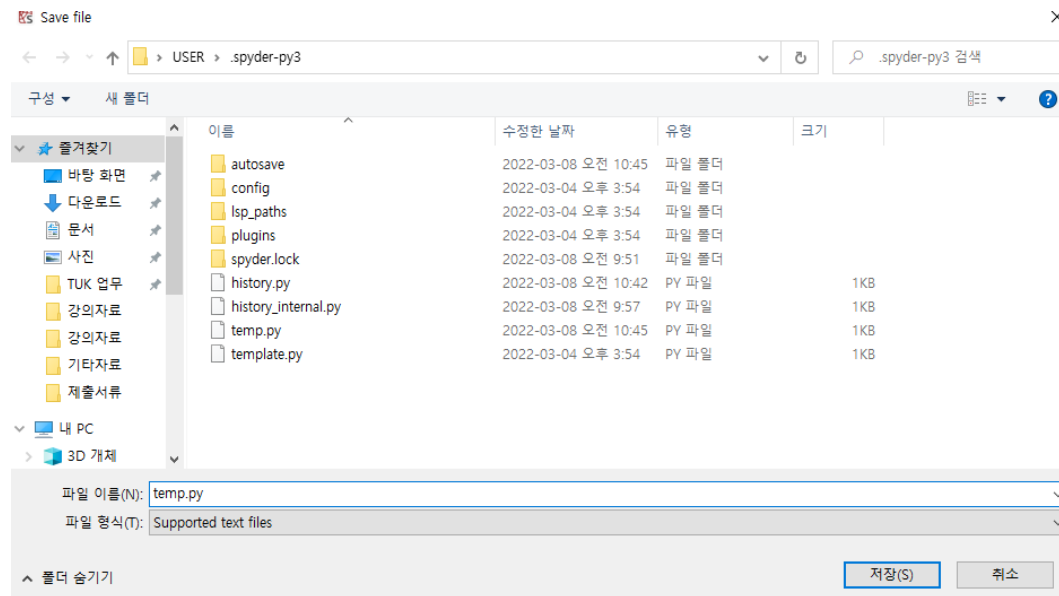
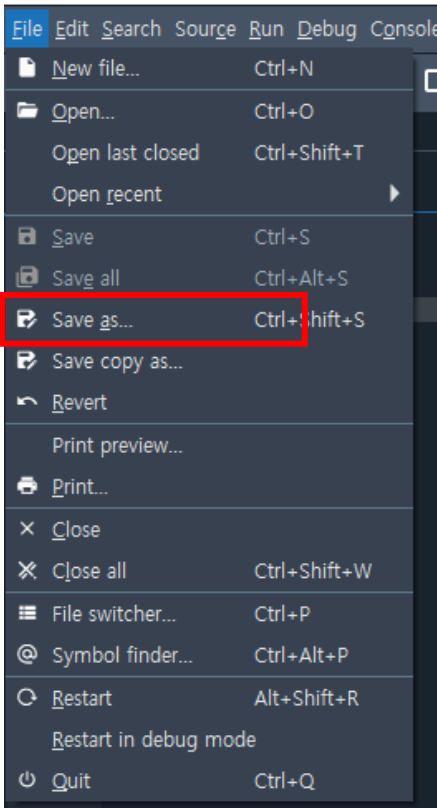


# Script 저장



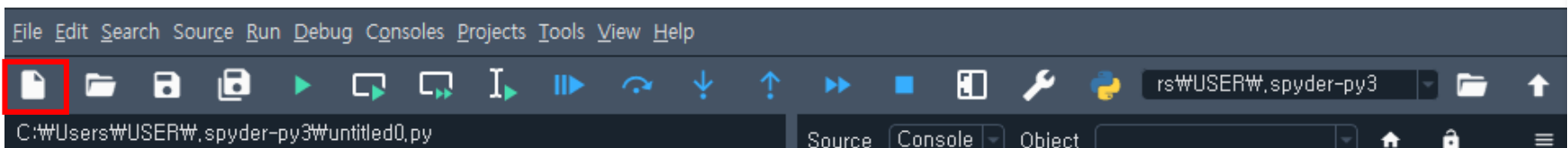
지능형 예측·진단 연구실  
Intelligent Prognostics & Diagnostics Lab.

Spyder (Python 3.9)



중요: 파일 이름은 항상 \*.py

Spyder (Python 3.9)



새로운 Script 파일

# Script 작성 시 유용한 기능: 주석 처리



지능형 예측·진단 연구실  
Intelligent Prognostics & Diagnostics Lab.

```
temp.py* x
1 print("Hello World 1")
2 print("Hello World 2")
3 print("Hello World 3")
4 print("Hello World 4")
5 print("Hello World 5")
```



Run

```
In [6]: runfile('C:/Users/USER/.spyder-py3/temp.py', wdir='C:/Users/USER/.spyder-py3')
Hello World 1
Hello World 2
Hello World 3
Hello World 4
Hello World 5
```

주석 처리 하고 싶은 곳을 지정하여 **Ctrl+1** (숫자 1), 주석 해제도 Ctrl+1

```
temp.py* x
1 print("Hello World 1")
2 # print("Hello World 2")
3 # print("Hello World 3")
4 print("Hello World 4")
5 print("Hello World 5")
6
```



Run

```
In [4]: runfile('C:/Users/USER/.spyder-py3/temp.py', wdir='C:/Users/USER/.spyder-py3')
Hello World 1
Hello World 4
Hello World 5
```

주석 처리 + 구분 하고 싶은 곳을 지정하여 **Ctrl+4** (숫자 1), 주석 해제는 Ctrl+5

```
temp.py* x
1 print("Hello World 1")
2 print("Hello World 2")
3 # =====
4 # print("Hello World 3")
5 # print("Hello World 4")
6 # print("Hello World 5")
7 # =====
8
```



Run

```
In [5]: runfile('C:/Users/USER/.spyder-py3/temp.py', wdir='C:/Users/USER/.spyder-py3')
Hello World 1
Hello World 2
```

# Variable (변수)



```
temp.py x
1 name = "Gil Dong"
2 age = 20
3 grade = "A"
4 weight = 70.1
```



Run

| Nam ▲  | Type  | Size | Value    |
|--------|-------|------|----------|
| age    | int   | 1    | 20       |
| grade  | str   | 1    | A        |
| name   | str   | 8    | Gil Dong |
| weight | float | 1    | 70.1     |

Help Variable Explorer Plots Files

int: 정수형, str: 문자열, float: 실수형

```
temp.py x
1 name = "Gil Dong"
2 age = 20.
3 grade = "A"
4 weight = 70.1
```



Run

| Nam ▲  | Type  | Size | Value    |
|--------|-------|------|----------|
| age    | float | 1    | 20.0     |
| grade  | str   | 1    | A        |
| name   | str   | 8    | Gil Dong |
| weight | float | 1    | 70.1     |

숫자 뒤에 . (점) 을 찍으면 실수형으로 적용

```
In [30]: type(name)
Out[30]: str
```

# Variable (변수)



```
1 name = "Gil Dong"
2 age = 20.
3 grade = "A"
4 weight = 70.1
5 print(name, "'s weight is", weight, "kg")
```



Run

```
Gil Dong 's weight is 70.1 kg
```

```
name = "Gil Dong"
age = 20.
grade = "A"
weight = 70.1
print(name, "'s weight is", weight, "kg")

weight = weight - 1
print(name, "'s weight is", weight, "kg")
```



Run

```
In [28]: runfile('C:/Users/USER/.spyder-py3/temp.py', wdir='C:/
Users/USER/.spyder-py3')
Gil Dong 's weight is 70.1 kg
Gil Dong 's weight is 69.1 kg
```

```
print(name + "'s weight is", weight, "kg")
```



Run

```
Gil Dong's weight is 69.1 kg
```

문자열 간 합쳐짐

- 비교 연산자
  - C언어와 동일

| 연산자 | 의미     |
|-----|--------|
| <   | 작다     |
| <=  | 작거나 같다 |
| >   | 크다     |
| >=  | 크거나 같다 |
| ==  | 같다     |
| !=  | 다르다    |

```
a,b,c=1,2,3  
print(a<b)  
print(a!=b)  
print(a==b)
```



Run

```
True  
True  
False
```

## ■ 수치 연산자

| 연산자      | 의미                       |
|----------|--------------------------|
| $x + y$  | x와 y의 합                  |
| $x - y$  | x와 y의 차                  |
| $x * y$  | x와 y의 곱                  |
| $x / y$  | x 나누기 y                  |
| $x // y$ | x 나누기 y 후 버림 적용 (정수 나누기) |
| $x \% y$ | x 나누기 y의 나머지             |
| $-x$     | x의 부호 반전                 |
| $+x$     | 변환 없음                    |
| $x ** y$ | x의 y제곱 (거듭제곱)            |

```
print("나누기 ", 5/2 )
print("정수 나누기 ", 5//2 )
print("나머지 연산 ", 5%2 )
print("거듭제곱 ", 5**2 )
```



```
나누기 2.5
정수 나누기 2
나머지 연산 1
거듭제곱 25
```

## ■ 수치 연산자 (함수형) <= 파이썬 기본 제공

| 연산자            | 의미                        |
|----------------|---------------------------|
| abs(x)         | x의 절대값, x의 크기(x가 복소수인 경우) |
| int(x)         | x를 정수형으로 변환               |
| float(x)       | x를 실수형으로 변환               |
| complex(x, y)  | 실수부 x, 허수부 y인 복소수, x+jy   |
| x.conjugate()  | 복소수 x의 켤레 복소수             |
| divmod(x, y)   | (x//y, x%y)               |
| pow(x, y)      | x의 y 제곱 (거듭제곱)            |
| round(x)       | x의 반올림                    |
| x.is_integer() | x가 정수이면 True              |
| x.real         | 복소수 x의 실수부                |
| x.imag         | 복소수 x의 허수부                |

```
print("절대값 ", abs(-3), abs(2+3j) )
print("복소수 ", complex(3, 5) )
print("켤레복소수 ", complex(3, 5).conjugate() )
print("거듭제곱 ", pow(5, 2) )
```

```
절대값 3 3.605551275463989
복소수 (3+5j)
켤레복소수 (3-5j)
거듭제곱 25
```

## ■ math 라이브러리

- C 표준 라이브러리 정의에 따름
- 라이브러리 적용을 위해선 “import xxx(라이브러리 명)” 을 먼저 선언해야 함.
- 주요 함수

```
math.sqrt(x)  
math.acos(x)  
math.asin(x)  
math.atan(x)  
math.cos(x)  
math.dist(x,y)  
math.sin(x)  
math.cos(x)  
math.pi  
math.e
```

```
import math  
print(math.sqrt(2))
```

```
import math as ma  
print(ma.sqrt(2))
```

라이브러리 명이 길 때, 약칭으로 선언 가능

tensorflow -> TF

**$e^2$ 은 어떻게 계산할 수 있을까?**

⇒ 두가지 이상 방법으로 표현해보세요

⇒ (Hint) 자동 완성 기능 참조



# Python 배열 1) List



## ■ list형 배열 만들기

- 배열 전체는 대괄호로 표시
- 각 원소는 콤마로 구분

```
a=[1, 2, 3, 4]  
A=[[1,2],[3,4]]  
print(a)  
print(len(a))  
print(A)  
print(len(A))
```



Run

```
[1, 2, 3, 4]  
4  
[[1, 2], [3, 4]]  
2
```

```
In [52]: print("A=",A,"\nlength=",len(A))  
A= [[1, 2], [3, 4]]  
length= 2
```

# Python 배열 1) List



## ■ list 메서드(method)

### ➤ Variable a에 대해서

| Method             | 의미                      |
|--------------------|-------------------------|
| <b>a.append(x)</b> | 리스트 a의 끝에 원소 x를 추가      |
| a.insert(i,x)      | 리스트 a의 i번째 위치에 원소 x를 추가 |
| a.remove(x)        | 리스트 a에서 원소 x를 삭제        |
| a.clear()          | 리스트 a를 삭제               |
| a.count(x)         | 리스트 a의 원소중 x와 같은 원소의 개수 |
| a.sort()           | 리스트 a의 원소 정렬            |
| a.reverse()        | 리스트 a의 원소 순서 반전         |

각 메서드에 대해서 실습해 봅시다.

insert나 remove를 사용하였을 때의 결과를 보면,  
가장 왼쪽의 배열의 index 번호는?

# Python 배열 1) List



- Index
  - 배열의 index는 0부터 시작
  - 개별 원소 접근
    - 리스트 이름 뒤에 대괄호를 이용

```
a = [1, 2, 3, 4, 5]
print(a[0], a[1], a[2], a[3], a[4])

b = [[1, 2], [3, 4], [5, 6], [7, 8]]
print(b[0][0], b[1][0], b[2][1])
print(b[0], b[1])
```



**Run**

```
1 2 3 4 5
1 3 6
[1, 2] [3, 4]
```

# Python 배열 1) List



## ■ Range

- 정수열(range) 생성 함수
- range(start, stop, step)
  - step이 없으면 1씩 증가
  - start가 없으면 0부터 stop-1까지 증가하는 정수열
  - start, stop, step 모두 정수여야 함

```
a=range(4,10,2)
print(a)
print(type(a))
print(list(a))
```



```
range(4, 10, 2)
<class 'range'>
[4, 6, 8]
```

Data type이 배열이 아니므로 list를 사용하여 변환

# Python 배열 1) List



## ■ 원소의 접근

### ➤ [start:stop:step]

- start부터 stop-1까지 step씩 증가하는 모든 주소의 원소

### ➤ [start:stop]

- start부터 stop-1까지 1씩 증가하는 모든 주소의 원소

### ➤ [start:]

- start부터 끝까지의 모든 원소

### ➤ [:stop]

- 처음부터 stop-1까지의 모든 원소

### ➤ [:]

- 처음부터 끝까지 모든 원소

```
a = list(range(1,10))  
print("a =", a)  
print(a[4:8])  
print(a[:8])  
print(a[4:])  
print(a[:])
```



```
a = [1, 2, 3, 4, 5, 6, 7, 8, 9]  
[5, 6, 7, 8]  
[1, 2, 3, 4, 5, 6, 7, 8]  
[5, 6, 7, 8, 9]  
[1, 2, 3, 4, 5, 6, 7, 8, 9]
```

# Python 배열 2) NumPy



- 행렬 연산을 위한 수학 라이브러리
  - 다차원 행렬 연산 가능
  - 데이터 처리에 용이
  - 가장 널리 사용되는 라이브러리
  - 공식 매뉴얼
  - `import numpy as np` 로 가장 많이 라이브러리를 적용시킴.



# Python 배열 2) NumPy



## ■ ndarray

| Nan ▲ | Type           | Size | Value       |
|-------|----------------|------|-------------|
| a     | Array of int32 | (5,) | [1 2 3 4 5] |

```
import numpy as np
a=np.array([1,2,3,4,5])
print(type(a))
```

```
<class 'numpy.ndarray'>
```

```
In [72]: a.dtype
Out[72]: dtype('int32')
```

dtype 지정 가능

```
a=np.array([1,2,3,4,5], dtype="int8")
print(a)
print(a.dtype)
```

```
[1 2 3 4 5]
int8
```

- 부호가 있는 정수 int(8, 16, 32, 64)
- 부호가 없는 정수 uint(8, 16, 32, 54)
- 실수 float(16, 32, 64, 128)
- 복소수 complex(64, 128, 256)

8비트 => 2의 8승 => 256개 표현 가능 => -128 ~ 127

```
a=np.array([127,128], dtype="int8")
print(a)
print(a.dtype)
```

```
[ 127 -128]
int8
```

128을 표현하는 비트가 없으므로 넘어감

```
b=np.array([[1,2],[3,4],[5,6]])
print(b)
```

```
[[1 2]
 [3 4]
 [5 6]]
```

행렬 표현 가능

# Python 배열 2) NumPy



## ■ Narray Method

| Method        | 의미                        |
|---------------|---------------------------|
| a.shape       | 배열 a의 형태를 알려줌, (행과 열의 길이) |
| a.size        | 배열 a에 들어있는 원소의 개수         |
| a.fill(x)     | 배열 a의 원소를 모두 값 x로 채움      |
| a.reshape()   | 배열 a의 형태를 바꿈              |
| a.transpose() | 배열 a에 transpose 연산 적용     |

특정 부분만 바꾸려면?  
=> 1d일 때, 2d일 때

```
import numpy as np

a = np.array( [1, 2, 3, 4, 5, 6] )
print("a =", a)
print("1. shape:", a.shape)
print("2. size:", a.size)
a.fill(3)
print("3. after fill:", a)
b = a.reshape(2,3)
print("4. after reshape:", b)
print("5. shape after reshape:", b.shape)
c = b.transpose()
print("6. transpose:", c)
print("7. shape after transpose:", c.shape)
```



Run

```
a = [1 2 3 4 5 6]
1. shape: (6,)
2. size: 6
3. after fill: [3 3 3 3 3 3]
4. after reshape: [[3 3 3]
 [3 3 3]]
5. shape after reshape: (2, 3)
6. transpose: [[3 3]
 [3 3]]
7. shape after transpose: (3, 2)
```



# Python 배열 2) NumPy



## ■ 유용한 Numpy 함수

| 명령                     | 의미                                            |
|------------------------|-----------------------------------------------|
| numpy.empty(shape)     | 지정된 shape의 빈 행렬, 실제로는 초기화하지 않으므로 무의미한 수가 들어있음 |
| numpy.eye(n)           | $n \times n$ 항등 행렬                            |
| numpy.ones(shape)      | 지정된 shape의 행렬, 모든 원소는 1로 초기화                  |
| numpy.zeros(shape)     | 지정된 shape의 행렬, 모든 원소는 0으로 초기화                 |
| numpy.full(shape, x)   | 지정된 shape의 행렬, 모든 원소는 x로 초기화                  |
| numpy.random.rand(a,b) | 랜덤 항을 가지는 $a \times b$ 행렬                     |

rand와 randn의 차이를 확인

```
import numpy as np
temp1=np.empty([3,3])
temp2=np.eye(3)
```



Run

| Name  | Type             | Size   | Value                                                       |
|-------|------------------|--------|-------------------------------------------------------------|
| temp1 | Array of float64 | (3, 3) | [[0.00000000e+000 0.00000000e+...<br>[0.00000000e+000 0 ... |
| temp2 | Array of float64 | (3, 3) | [[1. 0. 0.]<br>[0. 1. 0.]                                   |



더블클릭

|   | 0 | 1 | 2 |
|---|---|---|---|
| 0 | 1 | 0 | 0 |
| 1 | 0 | 1 | 0 |
| 2 | 0 | 0 | 1 |

# Python 배열 2) NumPy



- 유용한 Numpy 함수
  - `numpy.arange(start, stop, step)`
    - start부터 stop미만까지 step 간격으로 증가하는 숫자들의 배열
      - step이 없는 경우 자동으로 1씩 증가
      - start가 없는 경우 0부터 stop-1까지 증가
    - list의 range와 달리 실수형 사용 가능

```
a = np.arange(5, 10, 0.5)
b = np.arange(5, 10)
c = np.arange(10)
print("a =", a)
print("b =", b)
print("c =", c)
```



```
a = [5.  5.5 6.  6.5 7.  7.5 8.  8.5 9.  9.5]
b = [5 6 7 8 9]
c = [0 1 2 3 4 5 6 7 8 9]
```

- -10 부터 10까지 100등분한 배열을 만들어보세요.
  - arange 외에 더 간편한 함수가 있지 않을까?

# Python 배열 2) NumPy



## ■ 유용한 Numpy 함수

### ○ numpy.where(조건)

- ndarray에서 조건에 부합하는 원소들의 주소(index) 반환
- 반환값은 **tuple** 형임

수정할 수 없는 배열

```
import numpy as np

a = np.arange(5, 10, 0.5)
b = np.array(['A', 'A', 'B', 'B', 'B', 'C'])

id1 = np.where(a>7.0)
print(id1, type(id1))

id2 = np.where(b=='B')
print(id2, type(id2))
```



Run

```
(array([5, 6, 7, 8, 9]),) <class 'tuple'>
(array([2, 3, 4]),) <class 'tuple'>
```

어떤 상황에 활용할 수 있을까?

# Numpy를 활용한 행렬 연산



- 원소 단위 연산
  - 모든 연산자는 원소 단위 연산에 사용될 수 있음
  - 스칼라와 행렬의 연산

```
A = np.array([[1, 2, 3], [4, 5, 6]])  
print(A+1)  
print(A-2)  
print(2*A)  
print(A/2)  
print(1/A)  
print(2**A)
```



**Run**

```
[[2 3 4]  
 [5 6 7]]  
[[-1 0 1]  
 [ 2 3 4]]  
[[ 2 4 6]  
 [ 8 10 12]]  
[[0.5 1. 1.5]  
 [2. 2.5 3. ]]  
[[1.    0.5    0.33333333]  
 [0.25  0.2    0.16666667]]  
[[ 2 4 8]  
 [16 32 64]]
```

# Numpy를 활용한 행렬 연산



- 원소 단위 연산 (cont'd)
  - 행렬과 행렬의 연산
    - 동일 위치의 원소간의 연산이므로 행렬의 크기가 동일해야 함

```
A = np.array([1, 2, 3, 4, 5])  
B = np.full((5,), 2)  
print(A, B)  
print(A+B)  
print(A-B)  
print(A*B)  
print(A/B)  
print(A**B)
```



```
[1 2 3 4 5] [2 2 2 2 2]  
[3 4 5 6 7]  
[-1 0 1 2 3]  
[ 2 4 6 8 10]  
[0.5 1. 1.5 2. 2.5]  
[ 1 4 9 16 25]
```

# Numpy를 활용한 행렬 연산



- 행렬 연산 함수

| 함수                    | 의미 및 사용법                                           |
|-----------------------|----------------------------------------------------|
| <b>np.dot(a, b)</b>   | 두 행렬 a, b의 곱 (dot product)<br>a, b가 벡터인 경우 내적을 계산함 |
| np.vdot(a,b)          | 두 벡터 a, b의 내적                                      |
| <b>np.matmul(a,b)</b> | 두 행렬 a, b의 곱                                       |
| np.linalg.eig(a)      | 행렬 a의 eigenvalue와 eigenvector                      |
| np.linalg.norm(a)     | 행렬 또는 벡터 a의 norm                                   |
| np.linalg.det(a)      | 행렬 a의 determinant                                  |
| np.linalg.inv(a)      | 행렬 a의 역행렬                                          |
| <b>np.argmin(a)</b>   | 행렬 a의 최소값의 인덱스 검색                                  |
| <b>np.argmax(a)</b>   | 행렬 a의 최대값의 인덱스 검색                                  |
| <b>np.sum(a[])</b>    | 행렬의 원소 간 합                                         |

[간단실습] 0~10 사이의 랜덤값을 가지는 (10,10) 행렬을 만들고,  
1) 각 행 간의 원소합이 최대를 가지는 행의 index를 찾으시오.  
2) 각 열 간의 원소합이 최소를 가지는 열의 index를 찾으시오.

- matplotlib
  - Python을 위한 데이터 시각화 라이브러리
    - Matlab과 유사한 기능과 명령어 제공
    - 참조 사이트
      - <https://matplotlib.org/>
      - <https://wikidocs.net/92071>



# 그래프 출력 (matplotlib)



## ■ matplotlib

- Python을 위한 데이터 시각화 라이브러리
  - Matlab과 유사한 기능과 명령어 제공
  - 참조 사이트
    - <https://matplotlib.org/>
    - <https://wikidocs.net/92071>

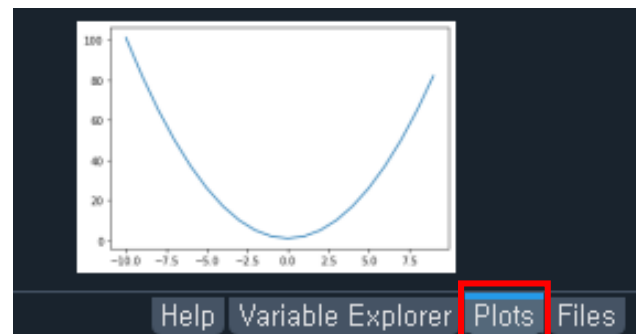
```
import numpy as np
import matplotlib.pyplot as plt

x=np.arange(-10,10,1)
y=x**2+1

plt.plot(x,y)
plt.show()
```



Run



Spyder에서는 입력 불필요

## ◦ 전용 그래프창을 새로 띄워 표시하는 방법

- Tools => Preference => Ipython console => graphics tab
- “graphics backend” section => “automatic” 선택



# 그래프 출력 (matplotlib)



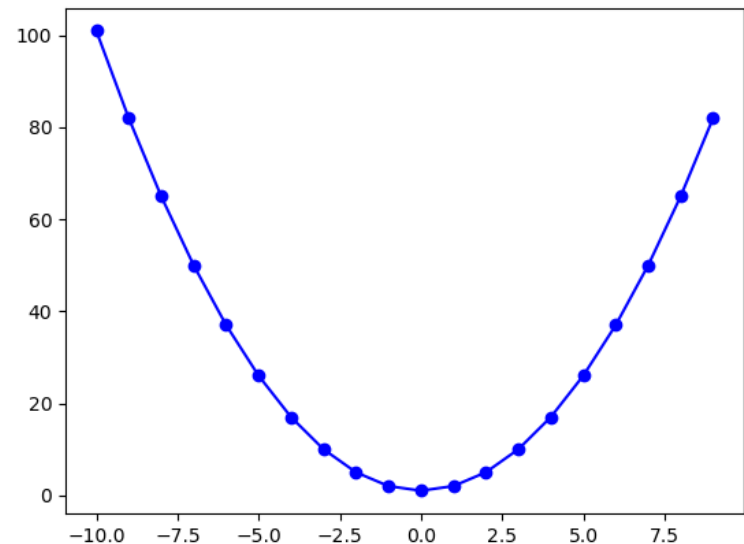
## ■ 그래프 스타일 지정

| character | description           |
|-----------|-----------------------|
| '_'       | solid line style      |
| '--'      | dashed line style     |
| '-.'      | dash-dot line style   |
| ':'       | dotted line style     |
| '.'       | point marker          |
| ','       | pixel marker          |
| 'o'       | circle marker         |
| 'v'       | triangle_down marker  |
| '^'       | triangle_up marker    |
| '<'       | triangle_left marker  |
| '>'       | triangle_right marker |
| '1'       | tri_down marker       |
| '2'       | tri_up marker         |
| '3'       | tri_left marker       |
| '4'       | tri_right marker      |
| 's'       | square marker         |
| 'p'       | pentagon marker       |
| '*'       | star marker           |
| 'h'       | hexagon1 marker       |
| 'H'       | hexagon2 marker       |
| '+'       | plus marker           |
| 'x'       | x marker              |
| 'D'       | diamond marker        |
| 'd'       | thin_diamond marker   |
| ' '       | vline marker          |
| '_'       | hline marker          |

- (예) 파란색, 실선, 동그라미 마커 => 'b-o'
- 속성의 순서는 무시
- 세 속성 중 일부만 지정하면, 나머지 속성은 자동 선택

| character | color   |
|-----------|---------|
| 'b'       | blue    |
| 'g'       | green   |
| 'r'       | red     |
| 'c'       | cyan    |
| 'm'       | magenta |
| 'y'       | yellow  |
| 'k'       | black   |
| 'w'       | white   |

```
plt.plot(x,y,'b-o')
```



# 그래프 출력 (matplotlib)



## ■ 그래프 스타일 지정

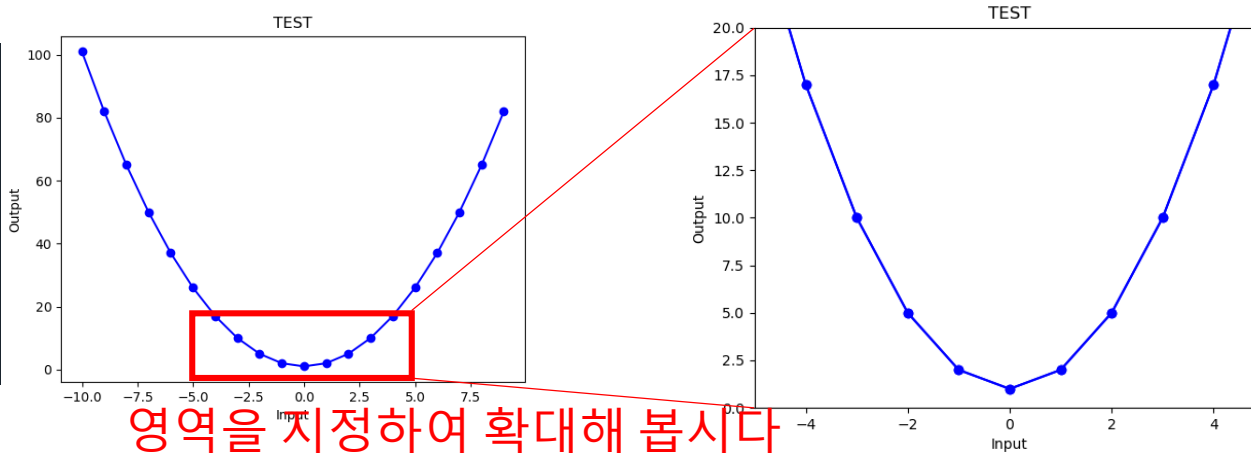
| Method           | 의미                    |
|------------------|-----------------------|
| plt.xlabel('문자') | X축 제목                 |
| plt.ylabel('문자') | Y축 제목                 |
| plt.xlim([a,b])  | X축을 a~b 구간으로 한정       |
| plt.ylim([a,b])  | Y축을 a~b 구간으로 한정       |
| plt.title('문자')  | 타이틀 지정                |
| plt.grid(True)   | 배열 a에 transpose 연산 적용 |

`plt.grid(True, axis='y', color='red', alpha=0.5, linestyle='--')`

```
import numpy as np
import matplotlib.pyplot as plt

x=np.arange(-10,10,1)
y=x**2+1

plt.plot(x,y,'b-o')
plt.xlabel('Input')
plt.ylabel('Output')
plt.title('TEST')
```



# 그래프 출력 (matplotlib)



## ■ 그래프 스타일 지정

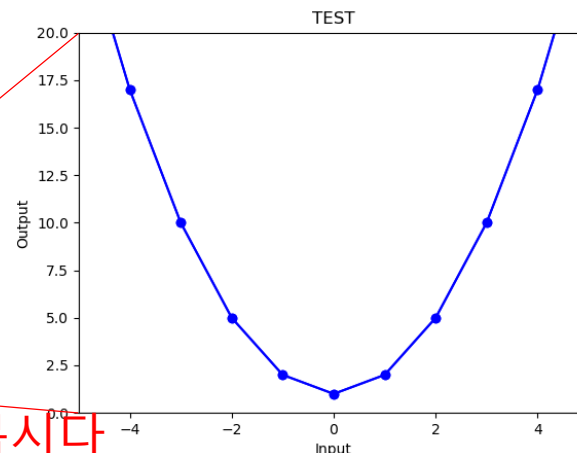
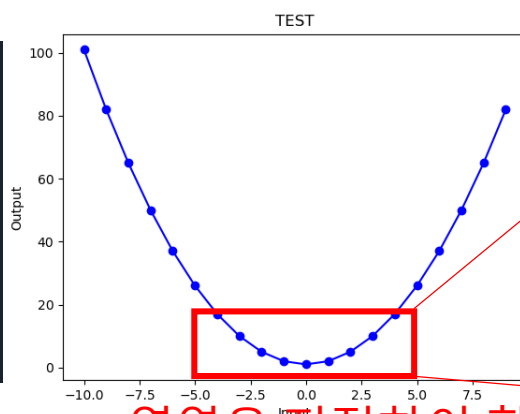
| Method           | 의미                    |
|------------------|-----------------------|
| plt.xlabel('문자') | X축 제목                 |
| plt.ylabel('문자') | Y축 제목                 |
| plt.xlim([a,b])  | X축을 a~b 구간으로 한정       |
| plt.ylim([a,b])  | Y축을 a~b 구간으로 한정       |
| plt.title('문자')  | 타이틀 지정                |
| plt.grid(True)   | 배열 a에 transpose 연산 적용 |

plt.grid(True, axis='y', color='red', alpha=0.5, linestyle='--')

```
import numpy as np
import matplotlib.pyplot as plt

x=np.arange(-10,10,1)
y=x**2+1

plt.plot(x,y,'b-o')
plt.xlabel('Input')
plt.ylabel('Output')
plt.title('TEST')
```



영역을 지정하여 확대해 봅시다

# 그래프 출력 (matplotlib)



## ■ 그래프 동시에 그리기

```
import numpy as np
import matplotlib.pyplot as plt

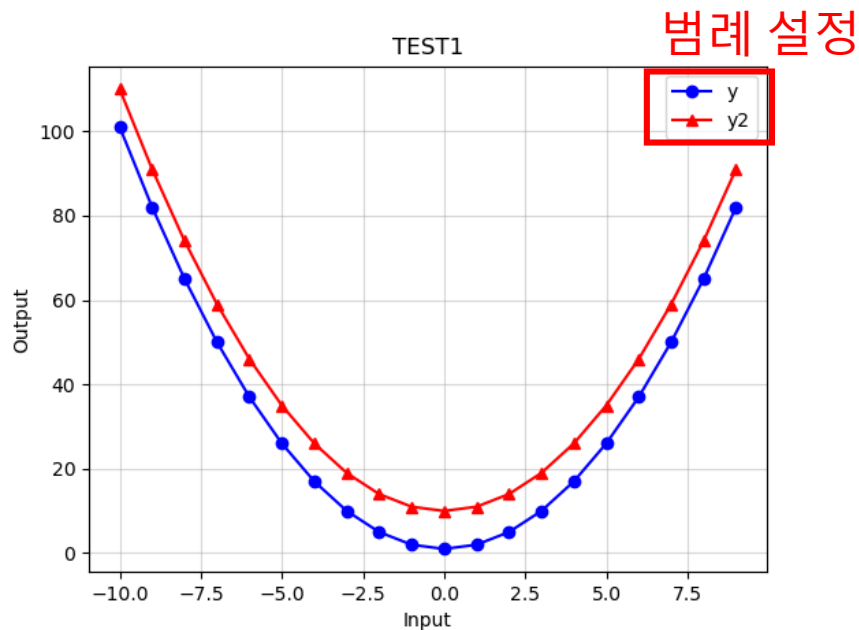
x=np.arange(-10,10,1)
y=x**2+1
y2=x**2+10

plt.plot(x,y,'b-o')
plt.plot(x,y2,'r-^')
plt.xlabel('Input')
plt.ylabel('Output')
plt.title('TEST1')
plt.grid(True,alpha=0.5)
plt.legend(['y','y2'],loc='upper right')
```

범례 설정



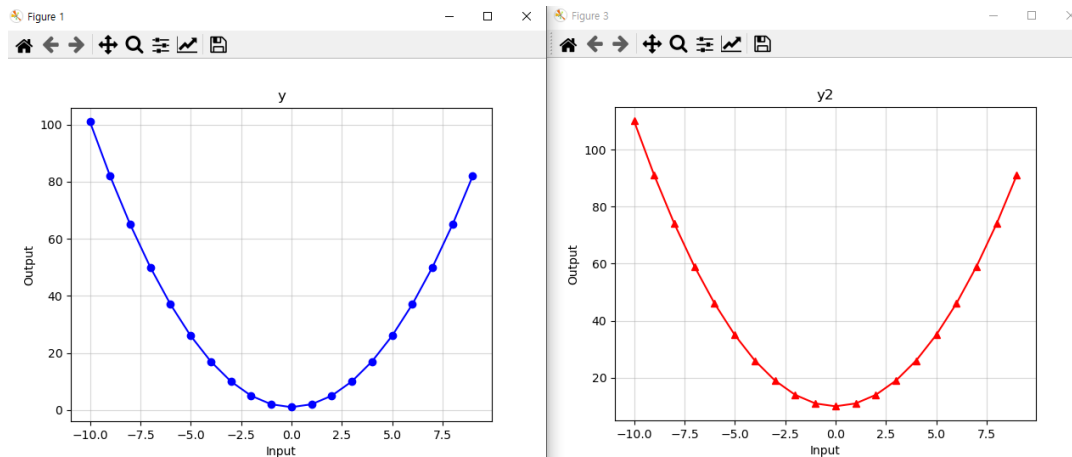
Run



## ■ 그래프 따로 그리기

```
plt.figure(1)
plt.plot(x,y,'b-o')
plt.xlabel('Input')
plt.ylabel('Output')
plt.title('y')
plt.grid(True,alpha=0.5)

plt.figure(3)
plt.plot(x,y2,'r-^')
plt.xlabel('Input')
plt.ylabel('Output')
plt.title('y2')
plt.grid(True,alpha=0.5)
```



# 그래프 출력 (matplotlib)

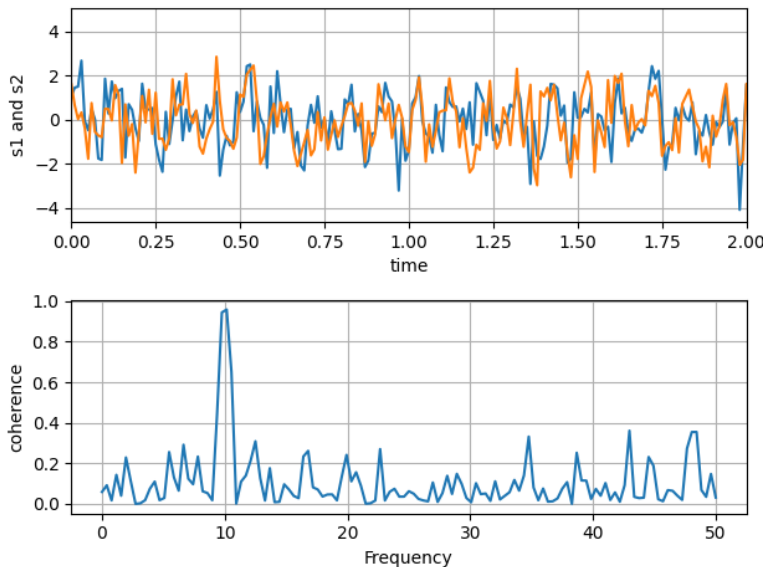


## ■ 그 외 활용 방법

- <https://matplotlib.org/> 에서 예제를 활용한다.

## Plotting the coherence of two signals

An example showing how to plot the coherence of two signals.



```
import numpy as np
import matplotlib.pyplot as plt

# Fixing random state for reproducibility
np.random.seed(19680801)

dt = 0.01
t = np.arange(0, 30, dt)
nse1 = np.random.randn(len(t))           # white noise 1
nse2 = np.random.randn(len(t))           # white noise 2

# Two signals with a coherent part at 10Hz and a random part
s1 = np.sin(2 * np.pi * 10 * t) + nse1
s2 = np.sin(2 * np.pi * 10 * t) + nse2

fig, axs = plt.subplots(2, 1)
axs[0].plot(t, s1, t, s2)
axs[0].set_xlim(0, 2)
axs[0].set_xlabel('time')
axs[0].set_ylabel('s1 and s2')
axs[0].grid(True)

cxy, f = axs[1].cohere(s1, s2, 256, 1. / dt)
axs[1].set_ylabel('coherence')

fig.tight_layout()
```

# 조건문 if



- 기본 문법

```
if 조건 1:  
    실행 루틴 1  
elif 조건 2:  
    실행 루틴 2  
elif 조건 3:  
    실행 루틴 3  
    ⋮  
else:  
    최종 실행 루틴
```

- if, elif, else로 구성
- 조건 판단 이후 콜론 (:) 사용
- 조건에 해당하는 실행 루틴은 들여쓰기로 구분

```
x = int( input("Enter one integer: "))  
  
if x < 0 :  
    print("Your number is negative.")  
elif x <= 10 :  
    print("Your number is in [0, 10].")  
else:  
    print("Your number is greater than 10.")
```



Run

```
Enter one integer: 3  
Your number is in [0, 10].
```

## • 기본 문법

```
for 변수 in 배열:  
    실행 루틴 1  
    실행 루틴 2  
    :
```

- 반복 횟수는 배열의 원소의 개수와 동일
- 변수는 배열의 원소를 나타냄
- for에 의해 반복되는 실행 루틴은 들여쓰기로 구분

```
a = ["Cat", "Dog", "Tree", "Flower", "Car"]  
  
num=0 ;  
for i in a:  
    num=num+1  
    print("Loop ", num)  
    print("Word:{0}, Length:{1} ".format(i, len(i)))
```



```
Loop 1  
Word:Cat, length:3  
Loop 2  
Word:Dog, length:3  
Loop 3  
Word:Tree, length:4  
Loop 4  
Word:Flower, length:6  
Loop 5  
Word:Car, length:3
```

- [간단실습] 0~10 사이의 랜덤값을 가지는 (10,10) 행렬을 만들고,  
1) 각 행 간의 원소합이 최대를 가지는 행의 index를 찾으시오.  
2) 각 열 간의 원소합이 최소를 가지는 열의 index를 찾으시오.

## • 데이터 쌓기

```
import numpy as np
temp=np.zeros([1,5])
for i in np.arange(1,3,1):
    temp[0,i]=i
    print(temp)
```

Indexing이 0 인 것에 주의

np.zeros([1,5])

|   | 0 | 1 | 2 | 3 | 4 |
|---|---|---|---|---|---|
| 0 | 0 | 0 | 0 | 0 | 0 |



Run

```
[[0. 1. 0. 0. 0.]
 [0. 1. 2. 0. 0.]
```

| Method      | 의미                 |
|-------------|--------------------|
| a.append(x) | 리스트 a의 끝에 원소 x를 추가 |

*append 를 사용해서 데이터를 쌓아보자*



# 반복문 while



## • 기본 문법

```
while (조건):  
    실행 루틴 1  
    실행 루틴 2  
    ⋮
```

○ 해당 조건이 참이면 실행, 거짓이면 종료

```
treeHit = 0  
while treeHit < 10:  
    treeHit = treeHit + 1  
    print("나무를 %d번 찍었습니다." % treeHit)  
    if treeHit == 10:  
        print("나무 넘어갑니다.")
```



```
나무를 1번 찍었습니다.  
나무를 2번 찍었습니다.  
나무를 3번 찍었습니다.  
나무를 4번 찍었습니다.  
나무를 5번 찍었습니다.  
나무를 6번 찍었습니다.  
나무를 7번 찍었습니다.  
나무를 8번 찍었습니다.  
나무를 9번 찍었습니다.  
나무를 10번 찍었습니다.  
나무 넘어갑니다.
```

## • break 활용

```
treeHit = 0  
while treeHit < 10:  
    treeHit = treeHit + 1  
    print("나무를 %d번 찍었습니다." % treeHit)  
    if treeHit == 5:  
        print("5회 찍었습니다 종료합니다.")  
        break
```



```
나무를 1번 찍었습니다.  
나무를 2번 찍었습니다.  
나무를 3번 찍었습니다.  
나무를 4번 찍었습니다.  
나무를 5번 찍었습니다.  
5회 찍었습니다 종료합니다.
```

가장 가까운 반복문에서 exit

# 사용자 정의 함수

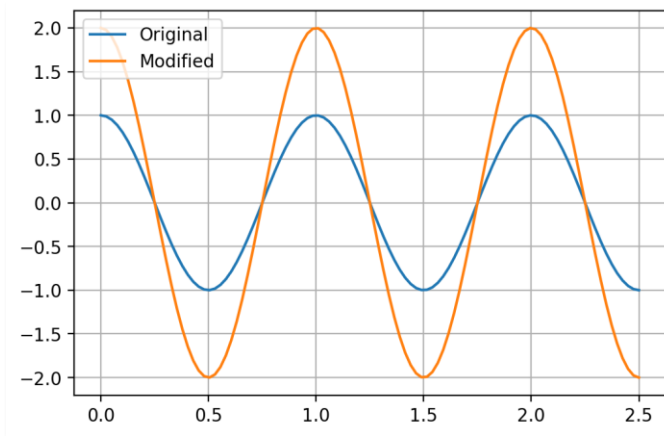


- 사용자의 필요에 따라 별도의 함수를 정의해 사용 가능
- 기본 문법

```
def 함수이름(입력변수1, 입력변수2, ...):  
    실행 루틴  
    ⋮  
    return 출력변수1, 출력변수2, ...
```

- 함수 선언의 위치는 py 파일 내 어디든 가능
  - 사용 위치는 선언 위치 이후
- 사용자 정의 함수의 실행 루틴은 들여쓰기로 구분
- 사용자 정의 함수 내부의 변수는 지역 변수로 취급

```
import numpy as np  
import matplotlib.pyplot as plt  
  
def op_times_two(x):  
    y = 2 * x  
    return y  
  
t = np.linspace(0, 2.5, 100)  
f = 1 ;  
y = np.cos(2*np.pi*f*t)  
y2 = op_times_two(y)  
  
plt.plot( t, y, t, y2 )  
plt.legend(['Original', 'Modified'])  
plt.grid(True)
```



- pandas

- 데이터 분석, 가공을 위한 파이썬 라이브러리
- 다양한 형태의 데이터 파일 입출력 함수 제공



```
import numpy as np
import pandas as pd
x = np.array([[1,2,3,4],[5,6,7,8]])
save_x=pd.DataFrame(x)
save_x.to_csv('C:\\test\\test.csv')
```

저장 경로 지정      파일명

|   | A | B | C | D | E |
|---|---|---|---|---|---|
| 1 |   | 0 | 1 | 2 | 3 |
| 2 | 0 | 1 | 2 | 3 | 4 |
| 3 | 1 | 5 | 6 | 7 | 8 |

header

index

```
import numpy as np
import pandas as pd
x = np.array([[1,2,3,4],[5,6,7,8]])
save_x=pd.DataFrame(x)
save_x.to_csv('C:\\test\\test.csv', index=False)
```

|   | A | B | C | D |
|---|---|---|---|---|
| 1 | 0 | 1 | 2 | 3 |
| 2 | 1 | 2 | 3 | 4 |
| 3 | 5 | 6 | 7 | 8 |

Index 파트 없이 저장을 원하면

- pandas.read\_csv

```
import numpy as np
import pandas as pd

temp_load=pd.read_csv('c:\\test\\test.csv')
```

파일을 받아오는 함수

| Index | 0 | 1 | 2 | 3 |
|-------|---|---|---|---|
| 0     | 1 | 2 | 3 | 4 |
| 1     | 5 | 6 | 7 | 8 |

- 변수.to\_numpy

```
import numpy as np
import pandas as pd

temp_load=pd.read_csv('c:\\test\\test.csv')
data_numpy=temp_load.to_numpy(dtype='int32')
```

Numpy array로 변환

temp\_load.columns.tolist() 를 사용해보자.

## ■ 이상치 검출 및 보정 알고리즘 만들기

1. -1초 ~ 1초 사이의 정현파 신호 생성
  - 크기: 10 V
  - 주파수: 60 Hz
  - 샘플링: 초당 30개
2. White Noise 생성 및 정현파 신호와 결합
  - 노이즈 피크 크기: 4 V
  - 샘플링: 초당 30개
3. 전압의 절대값이 12 V가 넘는 신호는 이상치로 판정 및 처리
  - 처리방법 1): 삭제
  - 처리방법 2): 0으로 대체
  - 처리방법 3): 전 후 값의 평균으로 보정
4. 이상치 보정 전 후 그래프 출력